Harden and Catch for Just-in-Time Assured LLM-Based Software Testing: Open Research Challenges

Mark Harman
Product Compliance and Privacy team, Meta Platforms
London, UK
University College London
London, United Kingdom

Peter O'Hearn
Fundamental AI Research team, Meta Platforms
London, UK
University College London
London, United Kingdom

Shubho Sengupta
Independent Software and AI Consultant
USA

ABSTRACT

Despite decades of research and practice in automated software testing, several fundamental concepts remain ill-defined and under-explored, yet offer enormous potential real-world impact. We show that these concepts raise exciting new challenges in the context of Large Language Models for software test generation. More specifically, we formally define and investigate the properties of hardening and catching tests. A hardening test is one that seeks to protect against future regressions, while a catching test is one that catches such a regression or a fault in new functionality introduced by a code change. Hardening tests can be generated at any time and may become catching tests when a future regression is caught. We also define and motivate the Catching 'Just-in-Time' (JiTTest) Challenge, in which tests are generated 'just-in-time' to catch new faults before they land into production. We show that any solution to Catching JiTTest generation can also be repurposed to catch latent faults in legacy code. We enumerate possible outcomes for hardening and catching tests and JiTTests, and discuss open research problems, deployment options, and initial results from our work on automated LLM-based hardening at Meta. This paper[1] was written to accompany the keynote by the authors at the ACM International Conference on the Foundations of Software Engineering (FSE) 2025.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**.

KEYWORDS

Software Engineering, Programming Languages, Large Language Models (LLMs).

ACM Reference Format:
Mark Harman, Peter O'Hearn, and Shubho Sengupta. 2025. Harden and Catch for Just-in-Time Assured LLM-Based Software Testing: Open Research Challenges. In *33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion '25), June 23–28, 2025, Trondheim, Norway*. ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3696630.3734199

[1]Author order is alphabetical. The corresponding author is Mark Harman.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
FSE Companion '25, June 23–28, 2025, Trondheim, Norway
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1276-0/2025/06
https://doi.org/10.1145/3696630.3734199

1 INTRODUCTION

For the past decade, at Meta, we (the authors) have been developing automated static and dynamic analysis and testing technologies. We have also been involved in the development of AI technologies, as a support to such analyses, and also as an independent technology in its own right. As well as our recent industrial experience, we have been active in the Software Engineering, Programming Languages, and Artificial Intelligence research communities over several decades. Our observation is that, despite much research and development, some of the most fundamental concepts that lie at the intersection between scientific research and practical software testing have yet to be adequately analysed, formalised, or defined, let alone fully tackled.

Research and practice are unequivocally and rapidly moving towards automated software test design, increasingly relying on language models to generate the test cases [26]. This makes it more important than ever to have crystal clear definitions of the most pressing problems facing practising software engineers, when it comes to automated test generation [11]. Automated test generation is not only important for testing code generated, both by humans and models, it may also prove useful for training and assessing the models themselves [50].

In this paper, we attempt to provide simple, intuitive, and precise formulations of the underlying concepts on which automated software test design rests. We first formalise the way in which generated tests harden against future regressions. This *regression* test generation has been the primary focus of both academic research and practice in automated test generation for the past few decades, with notable successes [6, 17, 36, 84]. However, as we show, it leads us into a Regression Only Trap, in which tests must pass on the current revision, and therefore cannot catch new bugs introduced by that revision, nor long-standing legacy bugs.

These regression tests are 'hardening' tests according to our terminology, because they seek to detect bugs in future changes rather than in the current system; they 'harden' against future regressions. Research and practice has tended to focus on regression testing due to the Oracle Problem [13]. Regression testing offers the automated test tool designer a readily deployable solution to the Oracle Problem: the Regression Oracle [7]. The Regression Oracle dictates that the behaviour of the previous version of the system should be preservered by any change, thereby ensuring that there are no regressions.

In Section 2 we briefly review our previous work on automated test generation at Meta, leading up to our current work on LLM-based automated unit test generation for hardening. The primary contributions of this work lie in the way we use LLMs and mutation.

Nevertheless, by remaining stuck in the Regression Only Trap, we (and the wider the community), have overlooked an important untackled problem: the Catching Just-in-Time test (Catching JiTTest) problem. A JiTTest is generated when a pull request is submitted. We also formalise and analyse JiTTests. By seeking to generate a catching JiTTest, we aim to catch bugs specifically *in* that pull request. A JiTTest is not concerned with some possibly distant future in which some regression may (or may not) occur, but the *immediate* future in which a pull request is about to land into production.

Generating Catching JiTTests allows us to escape the Regression Only Trap (ROT), but it represents a more challenging problem than generating hardening regression tests, although it subsumes that problem. It is also much more impactful. A JiTTest can catch bugs in new functionality *as well as* regressions, and it stops them literally *just-in-time*. That is, just before the pull request lands into production. Because they do not merely target regressions, JiTTests can *also* easily be repurposed to find *latent* bugs in the *existing* code base, as we show in Section 6.5. We believe the Catching JiTTest Challenge to be the most challenging and impactful problem in software testing. Although there has been initial discussion of just-in-time testing approaches [14, 28, 65, 81], much more remains to be done.

Section 3, introduces our formalisation of hardening and catching. Our formalism allows us to precisely determine when a hardening test subsequently catches a bug. It also allows us to distinguish between tests that catch regressions and those that catch bugs in new functionality. We use our formalism to precisely define the Catching JiTTest Challenge. Our definitions apply to testing in general, whether tests are generated by machine or by human, and irrespective of the generation technique used. Nevertheless, the motivation for our work is our recent development and deployment [5, 29] of LLM-based test generation (using an Assured LLMSE approach [8]).

The paper also includes a case-based analysis of possible deployments for hardening and catching tests, considering the balance of risk and reward inherent in their deployment. This analysis reveals new insights into deployment options.

The primary contributions of this paper are as follows:

(1) **Introduction of the Catching JiTTest Challenge**: Although challenging, we argue that the Catching JiTTest Challenge is the most highly impactful currently open challenge in automated test generation. We also show how advances in LLMs open the door to previously unavailable solutions, making the time ripe to tackle Catching JiTTest Challenge.
(2) **Precise Formal Foundations**: A formalisation of hardening and catching tests, and the distinction between regression catching and functionality catching tests.
(3) **Detailed Practical Deployment Analysis**: A case-by-case analysis of deployment options, illustrating how the formalism informs practical deployment options.

2 AUTOMATED TESTING AT META

In this section, we briefly review our previous and current work, at Meta, on automated test generation. The section is a brief review of work published in more detail elsewhere. The aim of the section is to make the paper self-contained and to motivate the need for the more detailed analysis of hardening and catching tests, introduced in Section 3.

2.1 Background

At Meta, we have previously deployed end-to-end test generation as well as unit-level test generation. Our end-to-end testing tools include Sapienz [60], deployed since 2017 [6] covering client-side testing on all platforms, and Fausta, deployed since 2021 [61], covering server-side testing for WhatsApp. The initial Sapienz deployment in 2017 used the implicit oracle [13], which has remained widespread throughout the company since 2017. More recently, we enhanced the Sapienz oracle to extend beyond the implicit oracle [79], and to cater for the rich diversity of user states needed to cover the state space [4].

These end-to-end testing systems can be thought of as simulation-based testing approaches [1], for which we have used Metamorphic Testing to test the simulation itself [2]. In traditional end-to-end testing, a test case simulates the end-to-end behaviour of the platform with respect to a single user. We have also deployed simulation-based test generation techniques that simulate the end-to-end behaviour of an entire interacting community of users. We call this approach 'social testing'.

Social testing is not only applicable to any platform in which users interact with each other, it is also necessary so that these interactions are fully tested [3, 79]. As we have shown in our previous work, social testing goes beyond traditional end-to-end testing, because it takes into account user 'personas', and the fact that the test oracle for one user differs (and may even *contradict*) the test oracle for another user.

These dynamic analyses are complemented by static analyses [23], such as Infer [18] and Zoncolan.

In addition to end-to-end test generation, we have also more recently developed unit-level test generation techniques using language models [5] and observations [9]. These unit test generation techniques focus on covering uncovered code.

To go beyond merely achieving additional structural code coverage, we also developed a mutation-guided approach [29]. The mutation-guided approach ensures that the generated tests find faults that no other test can find. This more recent mutation-guided approach generates hardening tests (according to Definition 6), because the tests are *guaranteed* to find at least one currently uncaught potential future fault.

2.2 Assured Large Language Model Based Software Testing (Assured LLMST)

Our approach to the generation of hardening tests is grounded in the principles of Assured LLM-based Software Testing (Assured LLMST). Assured LLMST is simply Assured LLM-Based Software Engineering (Assured LLMSE) applied to the field of Software Testing.

Assured LLMSE [8] is a generate-and-test approach to LLM-based software generation, inspired by the Genetic Improvement [69] approach to Search Based Software Engineering [37, 38, 73], also widely used in Software Testing [36, 63]. Assured LLMST is for Assured LLMSE, what Search Based Software Testing (SBST) is for Search Based Software Engineering (SBSE); it focusses on that subset of Software Engineering that is concerned with Software Testing.

Assured LLMSE addresses the two fundamental questions: How can we provide verifiable assurances that the LLM-generated code

(1) does not regress the properties of the original code ?
(2) improves the original in a verifiable and measurable way ?

2.3 Assured LLMST is Easier than Assured LLMSE

Fortunately Assured LLMS*T* is considerably less challenging, according to both these constraints, then LLMS*E*.

Does not Regress: In the context of software testing, not regressing the properties of the original code means not regressing the properties of the original test suite. For a test suite, the typical properties of interest lie in its ability to achieve a certain level of coverage and to catch a certain class of faults. When we add new test cases to an existing test suite (but do not remove any), we cannot, by definition, regress the test suite's ability to achieve coverage, nor its ability to reveal faults. Provided we treat each test case as a unique discrete unit (on which existing tests have no dependency), then adding additional test cases can only improve the overall test suite. This is in stark contrast to the situation in which we use language models to generate code *other* than tests. When generating code in general, it is easy to accidentally regress existing functionality.

Improves the Original: For the application to software testing, improving the original in a verifiable and measurable way is also simple: We can measure the additional coverage and/or can give an assurance that the new test finds a fault that is uncaught by any existing test. By contrast, the goal of improving code, *other* than test code, in a verifiable and measurable way can be challenging. For example, attempting to improve code execution time raises the challenges of measuring execution time reliably, over a distribution of potential inputs.

2.4 Online Assured LLMST

As with all machine learning [46], LLMSE can be either 'online' or 'offline' [8]. The extra time available to offline LLMSE typically allows the Assured LLMSE application greater opportunities to provide assurances.

One might naturally think of an 'online' approach as one that responds in *real* time. The archetype of online LLMSE is code completion. Code completion technologies, such as CoPilot [85] and CodeCompose [66], provide the generated code directly in the editor as the software engineer types text. This is clearly online because the response latency is the same as the duration of a keystroke.

When we formalise the notion of online, it becomes clear that there is, perhaps surprisingly, far more time available for some generated test deployment options than there is for code completion. The definition of 'online' (for the purposes of distinguishing between online and offline LLMSE) rests on the definition of 'real time', which has its own notion of timeliness [24, 55]. More formally, the LLMSE definition [8] of real time is repeated below as Definition 1.

Definition 1 (Real Time, taken from ref [8]). *A result is provided in* real time *for a consumer c and application a, if and only if c cannot perform a any better when the time spent awaiting the LLM response reduces.*

This is a 'utility'-based definition of real time. It focusses on the code being provided in *sufficient* time to be acted upon. Any JiTTest generation approach that contributes tests generated on the fly for a pull request merely has to generate these tests before the first human reviewer considers the pull request.

It may take a human reviewer hours or even days to provide that first review. According to a recent study [42], the median response time for Git projects was estimated at 1.75 hours. This estimate includes (instant response) static analysis bots (such as lint tools), which tends to make it an underestimate of the time available to the first human response. Even with this estimate, any test generated within 1.75 hours would thus be 'in time' for the review process with a probability of at least 0.5.

In fact, a more realistic estimate of the typical industry standard time for the first *human* response to a pull request is closer to 8 hours [62]. That is, there is an aspiration to respond within eight hours, and this is also therefore a realistic bound for 'real time' test generation (and thus for a JiTTest). As can be seen, the constraints for a test to be considered a JiTTest are not as tight as they might initially appear. This is important because eight hours affords a great deal of computational time that our tools and infrastructure can use to evaluate the assurances that can be offered to the engineer as part of the review process.

2.5 Initial Results on LLMST

At Meta, we recently developed and deployed a mutation-guided test generation system called ACH [29]. ACH ensures that the generated tests are genuinely hardening; they can catch a potential future bug. This should be contrasted with test generation that aims to improve structural code coverage but may not necessarily prove to be fault-revealing. Furthermore, since our deployment places test generation in a scenario in which the mutant is *guaranteed* to be killed, we also sidestep the familiar equivalent mutant problem [41, 51, 58, 68, 80].

The overall approach allows the ACH tool to give the following six assurances, as part of its overall Assured LLMSE approach.

(1) **Buildable**: The proposed new tests build, so are free from syntax errors and missing dependencies;
(2) **Valid Regression Tests**: The tests pass and do so consistently, so they are non-flaky [39] regression tests;
(3) **Hardening**: The new tests catch faults that *no* existing test can catch. Our mutation-guided tests are 'hardening' according to Definition 6. Furthermore, when an engineer accepts and lands such a generated test into production, the test becomes a Strong Hardening Test according to Definition 7.
(4) **Additional Coverage**: The tool automatically reports any additional coverage achieved.

(5) **Relevant**: Many of the new tests are closely coupled to the issue of concern;
(6) **Fashion Following**: Most of the new tests respect the coding style used by the existing tests available for the class under test; they are 'fashion followers' [5].

The first four of these six assurances are assurances in the strict meaning of the word: they are verifiable (and, therefore, falsifiable) guarantees that leave no room for doubt. The final two stretch the definition of 'assurance' a little bit; they are non-boolean aspirational attempts to provide improvement, in the sense of genetic improvement guarantees [69]. That is, we hope that the test will 'fashion follow', and therefore be more intuitive to the human reader, but this is something that can be somewhat subjective, rather like genetic improvement and restructuring [40, 71] or other aesthetic software improvements [72, 76]. We also hope the tests will be relevant, but this is also a somewhat subjective judgment.

We found that tests designed to be hardening (by killing mutants) that were deemed to be irrelevant by the engineer reviewing them, were more likely to be accepted when they improved coverage. Such tests proved acceptable even when they did not have another contribution [5, 29]. Of course, coverage improvement is not a necessary criterion for a test to be deemed acceptable because the test may reveal a fault without covering any new lines of code.

Therefore, we report additional coverage as a spinoff benefit; it is a measurable assurance we can track, but do not target. In managing software engineering deployment, it is important to distinguish between metrics that merit tracking from those that should become goals and Key Performance Indicators.

More details about our work on LLM-based test extension can be found in our FSE 2024 paper [5], while the more recent approach to incorporate mutation testing to guide the generation of tests toward those that harden with respect to specific faults (using mutation) can be found in our FSE 2025 paper [29].

3 FORMALISING HARDENING AND REGRESSION AND FUNCTIONALITY CATCHING TESTS

In this section we formalise the notions of hardening and catching tests using these to subsequently distinguish regression catching and functionality catching tests. The formalism allows us to make these distinctions more precise. In Section 5 we use it to analyse, case by case, the possible signals we may obtain from tests, the possible underlying causes, and consequent courses of action available when deploying hardening and catching test generation.

Software engineers typically deploy their code into a continuous integration pipeline, consisting of a sequence of 'revisions' (aka 'versions' or 'pull requests'). We purposely do not over-constrain the concept of a version. This could be a version that has been released, or one that is purely internal. The version may reside on a 'forked' branch or may reside on the 'trunk' of the commit tree. For the purposes of defining Just-in-Time software test generation, these details are irrelevant; we simply need a way to refer to the parent of a given code version.

Definition 2 (Fundamentals). *Defintion: We assume*

(1) *A tree repository with a function Parent(R) for a revision (aka pull request) R.*
(2) *A set Test Cases, T*
(3) *Predicates $\text{Builds}_R(t)$, $\text{Passes}_R(t)$, $\text{Fails}_R(t)$, for a revision R and test case t.*

The intuition behind these relations is as follows. When a test case, t builds on revision R this is denoted $\text{Builds}_R(t)$. A test case that builds can be executed. When executed, a test yields a signal that includes at least[2] two possible outcomes, pass and fail. When the outcome of executing test t on revision R is pass, this is denoted $\text{Passes}_R(t)$. When the outcome of executing test t on revision R is fail, this is denoted $\text{Fails}_R(t)$. A Test suite T is simply a set of test cases. Executing a test suite simply means executing each of the tests in the suite.

We make the simplifying assumption that tests can be executed independently. That is, the execution of one test from a suite does not influence the outcome of some other test from the suite. All of our definitions and concepts can be adapted to relax this independence assumption, but it makes the treatment more complex.

At first sight, it might seem that *all* tests should/would be buildable, but this is incorrect. When a new revision is created, some of the existing tests may become invalid (unbuildable). For example, if the name of a method is changed, then any test that relies on the method name will no longer build. Therefore, for each revision, not all tests will necessarily be buildable.

We are concerned with test timeliness, because we want to define just-in-time test generation. We therefore define a timely test, but do not require that it be executable, merely that it be available:

Definition 3 (Timely Test). *A timely test for the revision R, is available for attempted execution on R. That is, the test exists on or before the time at which the revision R is first submitted as a pull request for review.*

We want tests to be available when needed, but we also want to define a specific kind of test that is literally 'just-in-time' for a revision R:

Definition 4 (Just-In-Time Test (JiTTest)). *A Just-in-Time test, or 'JiTTest', for revision R, is a timely test for R that is not a timely test for Parent(R).*

A JiTTest is a kind of on-the-fly [28, 81] or just-in-time test [14, 65], in which we seek to construct a test as soon as the revision appears as a pull request. An engineer could write a JiTTest. Indeed, many engineers do write unit tests just-in-time, because they create them to accompany their pull requests.

In this paper, we are particularly interested in generated JiTTests, because this is an application of test generation that has been largely ignored in the research literature. The JiTTest may harden the pull request against future regressions (Hardening JiTTest) or may catch new bugs in it (Catching JiTTest). As we shall see, Catching JiTTest generation represents the most challenging, but also most impactful application area for automated test generation. We are

[2]There are often other possibilities such as timeouts, which may signify an engineering concern about incorrect test infrastructure operation. We do not want to constrain our definitions to include these signals, since they are engineering details that are irrelevant to understanding the fundamental signal of a correctly operating hardening or catching test.

also interested in timely hardening test generation. Although this is less challenging that Catching JiTTest generation, it is also useful, and represents the current deployment mode for most automated test generation, both in the research and practitioner communities.

We do not attempt to formally define 'generated test' for revision R, but our intuition is that a generated test is simply any test that is generated by machine, without human supervision.

3.1 Coverage

We want to be able to distinguish useful tests from useless (e.g., trivial) tests. For example, the trivial test `Assert(True)` is guaranteed to be both timely and buildable for any revision, according to our definitions, but clearly offers no value.

Coverage is currently the most widely used criterion for test generation in both academic research and industrial deployment. For example, there are automated test data generation systems based on Search Based Software Testing (SBST) [36, 63] such as Austin [56] for C and EvoSuite [31] for Java, and test generation systems based on concolic execution [17, 32, 75] such as Klee [16] that are publicly available and have been used both in research work and in practical deployment. These technologies focus on coverage achieved, seeking to increase coverage by making it a search objective, in the case of SBSE deployment, or making it the focus of a mixture of symbolic and concrete execution in the case of concolic execution systems.

Automated end-to-end test generation systems such as Sapienz [6, 60] and other simulation-based testing technologies, such as Meta's WW [3] and Fausta [61], and Google's RecSim [48] also increase coverage, but as a side benefit of seeking to catch bugs. These end-to-end systems target the detection of bugs rather than the elevation of coverage. They tend to rely on the so-called implicit oracle [13] or the regression oracle [7], so they cannot target the revelation of faults in new functionality.

Automated fuzzing techniques [59], such as AFL [84] have also been proven to be successful in industrial deployment, due to their conceptual simplicity, applicability and ability to catch security flaws. These technologies also tend to focus on increasing the coverage of the system under test and use the implicit oracle, traditionally focusing on features that, in any and all applications, could lead to a security flaw.

The implicit oracle captures that which no software system should ever do (such as crashing and running out of memory), rather than the properties that we expect for a specific system (such as its functional and other behavioural properties). This means that implicit oracle tests can only target a very restricted, albeit highly egregious, class of bugs.

Sadly, any coverage-improving test may simply execute code that does not matter while chasing the goal of elevating coverage. The freshly covered code may not matter because, for example, it is never executed by any users. Even when a new test covers previously uncovered code that *does* matter, it may not be executed in a way that can reveal bugs. It is well known, from many empirical studies, that increasing coverage does not provide a guarantee of increasing fault revelation [30, 33, 49, 52]. In fact, evidence tends to suggest [19] that the best techniques to increase fault revelation focus specifically on *faults* targeted by mutation testing [51].

These empirical observations are also born out by our definitions. A test can increase coverage, yet be guaranteed, by construction, to never reveal *any* bugs. For example, consider a test that executes additional lines of code yet relies on an oracle (assertion) that is semantically equivalent to `Assert(True)`. This is a rather extreme case of the well-known coverage dilemma, but it illustrates the fundamental underlying problem. To address this, we define a 'hardening' test case; one that hardens the code against future changes by catching *at least one bug* or type of bug in a well-defined way.

3.2 Oracles

In order to define a hardening test case, we first need to introduce the concept of an oracle [13]. Strictly speaking, we do not require a full oracle, but merely a partial oracle that is able to determine the outcome for each test case on each revision. This is an important distinction, because we do not necessarily require a full specification of the system, but merely an answer to the question: 'should test t have passed or failed on revision R?'

Definition 5 (Oracle). *Given a test t and a revision R, Oracle $Oracle_R(t)$ gives the expected result for test t in R.*

t *is a* **true positive** *for R if* $Fails_R(t) \wedge Oracle_R(t) = \texttt{fail}$.
t *is a* **false positive** *for R if* $Fails_R(t) \wedge Oracle_R(t) = \texttt{pass}$.
t *is a* **true negative** *for R if* $Passes_R(t) \wedge Oracle_R(t) = \texttt{pass}$.
t *is a* **false negative** *for R if* $Passes_R(t) \wedge Oracle_R(t) = \texttt{fail}$.

We leave the question of how one determines the oracle deliberately unspecified. In much of the literature on oracles [13] there is an implicit assumption that there exists a *single* ground-truth oracle for a software system, albeit one which may be unknown or only partially specified, but this is an unreasonable assumption. This was pointed out as early as 1979 with the publication of a seminal paper [21]. That paper was deemed, by some, to be controversial [22] at the time. Nevertheless, it has become increasingly apposite as the nature of Software Engineering has evolved over the intervening years [44, 45, 67]. Specifically, Software Engineering is a partly social process, so one engineer's (and/or one user's) oracle may differ from another.

We do not wish to enter into this detail here so we assume that, given a revision and a test, there is agreement on the oracle for the expected behaviour of the test on that revision. For example, for practical purposes, we may consider that the decision is confined to the author of the pull request, since this is the person (or machine) who, for all practical purposes, will play the role of deciding whether a test signal is acceptable.

3.3 Hardening Tests

In this section, we formally define the concept of a hardening test: one that is designed to pass on the current version of the system and catch future regressions.

This is the kind of test produced by many test generation systems and approaches, including Symbolic Execution systems [17], such as Klee [15], CUTE [75] and DART [32], Search Based Systems such as EvoStuite [31] and Austin [56], Mutation-based test generation Systems such as Javalanche [74] and SHOM [35], Replay Systems such as Fausta [61] and JRapture [77], Observation-based systems

such as Carving [25], TestGen [9] and Pankti [78] and LLM-based systems such as TestGen-LLM [5], CoverUp [70], HITS [83] and ACH [29].

Although these systems have different notions of coverage, such as mutants, serialised object observations and code, they all share the same approach to the oracle, which is that the behaviour of future changes should not disrupt the previously observed behaviour. This is the 'regression oracle' [7], in which a previous revision of the system acts as the ground truth oracle for future revisions.

A test t for the revision R, is a *weak hardening test*, denoted $\text{Hardening}_R(t)$, if and only if it passes on R and there exists a possible next revision on which it fails when it should. More formally, Definition 6 defines (weak) hardening.

Definition 6 ((Weak) Hardening Test). $Hardening_R(t) \Leftrightarrow Passes_R(t) \wedge \exists R'.Parent(R') = R \wedge Fails_{R'}(t) \wedge Oracle_{R'}(t) = \texttt{fail}$.

Definition 6 is 'weak' in the sense that it does not require that the test, t should correctly pass on R, merely that it should correctly fail on some future revision, R'. The test 'hardens' the code base against at least one buggy revision it might encounter in future.

A test t for revision R, is a *strong hardening* test denoted $\text{SHardening}_R(t)$, if it is a weak hardening test for R and it passes correctly (true negative) for R. More formally, Definition 7 defines strong hardening.

Definition 7 ((Strong) Hardening Test).

$$\begin{gathered} SHardening_R(t) \Leftrightarrow Hardening_R(t) \wedge Oracle_R(t) = \texttt{pass} \\ \Leftrightarrow \\ Passes_R(t) \\ \wedge \\ Oracle_R(t) = \texttt{pass} \\ \wedge \\ \exists R'.Parent(R') = R \\ \wedge \\ Fails_{R'}(t) \\ \wedge \\ Oracle_{R'}(t) = \texttt{fail} \end{gathered}$$

A strong hardening test is one for which we know that the passing signal on the current revision is a correct passing signal. In general, this relies on oracle knowledge which cannot always be known at test generation time; it may not even be well defined, since engineers may disagree, with the consequence there is no ground truth. Nevertheless, if there is a known agreed oracle outcome for the test case, then it would be possible to determine, at test generation time, whether a weak hardening test is strong. Indeed, in many cases it is worth the additional (minor) friction of asking the engineer to check that the test is correct in passing on the current revision. This human-in-the loop check ensures that a weak hardening test is also a strong hardening test, thereby allowing us to proceed with greater confidence.

This is what we have done with our deployment of TestGen-LLM [5] and ACH [29], both of which generate tests using LLMs, and propose these to the engineer who checks that the behaviour of passing on the current version of the system is correct. In the case of TestGen-LLM the engineer also has to check whether the test will likely catch future regressions. This extra obligation places a further burden on the human engineer that is removed by ACH, since it automatically provides an example of a fault that is *guaranteed* to be caught by the new proposed test. As a result, ACH tests accepted by the engineer are strong hardening tests; the engineer confirms that the test correctly passes on the current revision, while the ACH technology automatically generates the assurance that it catches at least one bug (the mutant) not caught by any existing test.

A strong hardening test is also a weak hardening test, by definition, but not vice versa. When we refer to a 'hardening test' (without qualification) we mean a weak hardening test (that is, one that may also happen to be strong). When we wish to refer to a *weak* hardening test that is also *not a strong* hardening test, we call it a '*strictly* weak hardening' test.

Notice that a hardening test may be a coverage-improving test for some structural coverage criterion, such as statement coverage, but not necessarily so. The test may cover exactly the same code as already covered, yet do so in a different way to any existing test.

A hardening test distinguishes the behaviour of the current version of the system from that of some future change. In the terminology of mutation testing, where we think of a buggy R' as a mutant of R, a hardening test *kills* the mutant. It was this desire to go beyond coverage to automatically generate hardening tests that led to our recent work on mutation-guided LLM-based test generation [29]. ACH deployment guarantees, not only that tests are strong hardening, but also that they can catch at least one bug that is uncaught by any existing test.

Running example: Throughout the following definitions we will present illustrative examples of tests and pull requests, based on a hypothesised simple database application. This illustrative application involves a 'product' object that includes a list of supplier locations. This list should be initially empty. It should be a list type because there could be more than one supplier, and it should be empty initially because we do not initially have any suppliers when we create the object. Example 1 gives a unit test that we might write for this database application.

Example 1 (Weak Hardening Test).

```
def weak_check_initial_supplier_locations():
  product = Product()
  locs = product.get_supplier_locs()
  assertTrue (not locs)
```

If a subsequent pull request were to introduce a bug that inadvertently assigns a nonempty list to the locations field on initialisation, Example 1 would catch that bug. However, suppose the constructor `Product()` *already* contains a bug (in the current revision) that allows the supplier locations field to be null (i.e., `None` type in Python). In this situation, Example 1 is incorrectly passing on the current revision, due to the familiar problem in Python that the unary `not` operator returns `True` when applied to the empty list, but also when applied to the null value `None`.

If the current revision contains this bug, then Example 1 is a *strictly weak* hardening test (i.e., it is not strong hardening). We can create a better test, Example 2 below, which is strong hardening. Example 2 fixes the assertion, replacing `assertTrue (not locs)` with `assertTrue (locs is not None and len(locs) == 0)`.

Example 2 (Strong Hardening Test).

```
def strong_check_initial_supplier_locations():
  product = Product()
  locs = product.get_supplier_locs()
  assertTrue (locs is not None and len(locs) == 0)
```

Strictly weak hardening tests are problematic because they introduce a false sense of security; the current revision is wrongly believed to be more correct than it is. Furthermore, the same test weakness will extend to testing future possible revisions that may contain the same bug, and which the strictly weak test would allow to slip through as an *avoidable* false negative. This is why we believe it is worth the minor friction inherent in proposing newly generated weak hardening tests to engineers to *review* (rather than simply automatically landing them into production without human review).

The engineers' reviews ensure that we check the oracle and make the engineers the final arbiters of whether tests should land into production. With this simple approach, we significantly increase the probability that the weak hardening tests we generate will prove to be *strong* hardening tests when deployed.

3.4 Hardening Tests' Precision and Recall

A test t for revision R, is a *Perfect Precision Hardening* test denoted $\mathsf{PPHardening}_R(t)$, if it is a hardening test for R and, on any occasion when it fails, it is right to fail according to the oracle (no false positives). More formally, Definition 8 defines a (Perfect Precision) Hardening Test.

Definition 8 ((Perfect Precision) Hardening Test).

$$\begin{gathered} \mathit{PPHardening}_R(t) \Leftrightarrow \mathit{Hardening}_R(t) \\ \wedge \\ \forall R'.\mathit{Fails}_{R'}(t) \Rightarrow \mathit{Oracle}_{R'}(t) = \texttt{fail} \end{gathered}$$

A test t for revision R, is a *Perfect Recall Hardening* test denoted $\mathsf{PRHardening}_R(t)$, if it is a hardening test for R and, on any occasion when it should fail according to the oracle, it does fail (no false negatives). More formally, Definition 9 defines a (Perfect Recall) Hardening Test.

Definition 9 ((Perfect Recall) Hardening Test).

$$\begin{gathered} \mathit{PRHardening}_R(t) \Leftrightarrow \mathit{Hardening}_R(t) \\ \wedge \\ \mathit{Oracle}_{R'}(t) = \texttt{fail} \Rightarrow \forall R'.\mathit{Fails}_{R'}(t) \end{gathered}$$

We can conjoin definitions 8 and 9 to produce a definition of a 'perfect precision and recall hardening test'. A perfect precision and recall hardening test passes on the current revision, and correctly fails on every possible future buggy revision (and fails on no other revision). That is, for every possible future revision, the test fails if and only if the revision is buggy with respect to the property tested.

A perfect precision and recall test is an idealised test that may be hard to achieve in practice. Example 3 illustrates how hard it is to achieve perfect precision and recall. It defines a scenario in which a test may have *almost* perfect precision and recall. Example 3, is a test that might, at first, appear to have perfect precision and recall. It is certainly *hard* to construct an example where it incorrectly fails, or an example where it incorrectly passes, but not impossible.

Example 3 (Near perfect precision and recall test: `check_foo`). *Suppose the function* `foo` *has a single nullable formal parameter but its implementation assumes that the actual parameter passed is never null. For this example, we then write a unit test,* `check_foo` *that fails precisely when a call site to* `foo` *passes a null actual parameter.*

As the pull request in Example 4 below shows, it is almost impossible for a test to attain perfect precision and recall. Example 4 does not challenge the belief that Example 3 exhibits perfect recall, but it makes it clear that it does not exhibit perfect precision. Once the pull request is submitted, it would become clear the `check_foo`'s failure when the call site passes null to `foo`, is no longer a correct failure; it is a false positive. Since the revision in Example 4 exists (as a potential future revision), we must conclude that the precision of `check_foo` from Example 3 never *was* perfect.

Example 4 (A pull request that confirms that check_foo does not have perfect precision). *Consider a pull request for the function* `foo` *that adds a check on the actual parameter, and gracefully behaves reasonably when that parameter should ever be null.*

3.5 Measuring Tests' Precision and Recall

Although we cannot expect tests to have perfect precision and recall, we can *measure* the precision of a hardening test over a series of test outcomes. When measuring precision we tend to use pseudo false positives [6] as our criterion, because when an engineer rejects a test signal, even if it is a 'correct' signal, then this represents the degree of friction, and that should be regarded as a kind of false positive; a pseudo false positive. The distinction between pseudo-false positive and false positive is an interesting illustration of the difference between practical software testing and theoretical software testing.

We can measure precision, but tests' recall cannot be directly measured, because it relies on perfect knowledge about the ground-truth expected behaviour. Instead, we can *estimate* the recall using, for example, mutation testing (to sample the space of potential faults) or leak through of faults into production (to determine a lower bound on the number of false negatives).

Practicing software engineers are typically forced to care more about precision than recall. Perfect recall is unachievable, because perfect (i.e., exhaustive) testing is unachievable, and so recall is an aspiration. Perfect precision is also unachievable, because the underlying questions are fundamentally undecidable in general. Although we cannot hope achieve *perfect* precision and recall, *unnecessarily* low precision can lead to the entire automated testing technology being abandoned due to friction on the engineers who waste their time considering false positives. By contrast, unnecessarily low recall is less risky to deployment. In this section, we want to capture this inherent prioritization, and to suggest a different way of measuring precision and recall.

It is important to avoid measuring and goaling on precision and recall independently. It is also important to avoid aggregates such as the F1 score that implicitly treat the two measurements as convertible commodities. That is, by aggregating, we implicitly accept that it is possible to trade a certain amount of precision degradation for a certain amount of recall improvement. This does not carry over well to the use case of industrial deployment of automated test generation.

Rather, we propose to goal on the $R@P = p$. That is, the recall achievable when the precision (P) is fixed to be no lower than p. For example, $R@P = 0.8$ is the recall achievable when we demand

a precision of at least 0.8, thereby ensuring that the ratio of false positives to true positives is no worse than 1:4.

Choosing 0.8 for p is a conservative choice. We can be fairly sure that, for most applications, this will be a sufficiently high precision: engineers will be prepared to tolerate friction 20% of the time, if the other 80% of cases throw up true positive catches.

3.6 Catching Tests

In this section, we define the notion of a catching test: one that fails on the current revision and may thereby reveal a bug.

A *weak catching* test t, for revision R, denoted $\mathsf{Catching}_R(t)$, is simply a test that fails on R. More formally, Definition 10 defines a (Weak) Catching Test.

Definition 10 ((Weak) Catching Test).

$$Catching_R(t) \Leftrightarrow Fails_R(t)$$

A weak catching test on some revision R may give a false positive signal, so we define the concept of a strong catching test (that gives a true positive signal). A *strong catching* test t, for revision R, denoted $\mathsf{SCatching}_R(t)$, is a weak catching test for which the failure on R is regarded as a true positive according to the oracle. More formally, Definition 11 defines a (Strong) Catching Test.

Definition 11 ((Strong) Catching Test).

$$SCatching_R(t) \Leftrightarrow Fails_R(t) \wedge Oracle_R(t) = \texttt{fail}$$

Example 5 is a pull request for which the hardening test from Example 2 would subsquently also become a (strong) catching test.

Example 5 (Strong Catching Test). *Suppose a pull request inadvertently updates the code for the* `Product` *constructor to set the supplier location to* `None`.

The test defined in Example 2 will correctly fail on the pull request in Example 5, so we conclude that the test is a hardening and (strong) catching test for the pull request in Example 5.

As with weak and strong hardening tests, we refer to a weak catching test that is also not a strong catching test as a '*strictly* weak catching' test.

In general, Search Based Software Testing (SBST) [36, 63] systems such as Austin [56] for C and EvoSuite [31] for Java, and test generation systems based on concolic execution [17, 32, 75] such as Klee [16] generate hardening tests. They seek to catch future regressions, but use the current version as the oracle, so cannot catch new bugs, just regressions.

By contrast, automated fuzzing techniques [59], such as AFL [84] and automated end-to-end test generation systems, such as Sapienz [6, 60], WW [3] Fausta [61], and Google's RecSim [48] can potentially find Catching JiTTests. However, they use the implicit oracle (not the regression oracle) to determine whether the test has failed. These Catching JiTTests can therefore catch, only those truly egregious and obvious faults that are flagged by the implicit oracle (such as memory violations and crashes).

3.7 Regression Catching Test

It could be that a catching test for R is also a hardening test for the parent of R. We call this a 'regression catching' test, because a hardening test is a regression test; the current behaviour of the system acts as an oracle (the 'regression oracle' [7]).

A test t for revision R is a *Weak Regression Catching Test* for R, $\mathsf{RCatching}_R(t)$, if it is a weak catching test for R and is also a hardening test for the parent of R. More formally, Definition 12 defines a Weak Regression Catching Test.

Definition 12 (Weak Regression Catching Test).

$$\begin{gathered} RCatching_R(t) \\ \Leftrightarrow \\ Passes_{Parent(R)}(t) \\ \wedge \\ (\exists R'.\, Parent(R) = Parent(R) \wedge Fails_{R'}(t) \wedge Oracle_{R'}(t) = \texttt{fail}) \\ \wedge \\ Fails_R(t) \end{gathered}$$

A test t for revision R is a *Strong Regression Catching Test* for R, $\mathsf{SRCatching}_R(t)$, if it is a strong catching test for R and is also a hardening test for the parent of R. More formally, Definition 13 defines a Strong Regression Catching Test.

Definition 13 (Strong Regression Catching Test).

$$\begin{gathered} SRCatching_R(t) \\ \Leftrightarrow \\ Passes_{Parent(R)}(t) \\ \wedge \\ (\exists R'.\, Parent(R) = Parent(R) \wedge Fails_{R'}(t) \wedge Oracle_{R'}(t) = \texttt{fail}) \\ \wedge \\ Fails_R(t) \\ \wedge \\ Oracle_R(t) = \texttt{fail} \end{gathered}$$

This simplifies to

$$SRCatching_R(t) \Leftrightarrow Passes_{Parent(R)}(t) \wedge Fails_R(t) \wedge Oracle_R(t) = \texttt{fail}$$

Since the test defined in Example 2 is both hardening and (strong) catching for the pull requests in Example 5, we conclude that it is a (strong) regression catching test for this pull request.

3.8 Functionality Catching Test

A catching test does not necessarily need to be a regression catching test. We want to push testing beyond the Regression Only Trap (ROT). The revision for which the test catches a bug may have intended new functionality which the test catches. If this is new functionality, then the catching test should not pass on the parent. It may even be that the catching test, t for a revision, R does not even build on $\mathsf{Parent}(R)$. For example, suppose R introduces a new method, m and that t tests the behaviour of m. Since m does not exist in $\mathsf{Parent}(R)$, then t will not build on $\mathsf{Parent}(R)$.

We define a Functionality Catching test, as one that catches faults in R but is not a hardening test for $\mathsf{Parent}(R)$. As with regression testing, it can be strong or weak depending on whether the failure on R concurs with the oracle.

A *Weak Functionality Catching* test t, for revision R is a weak catching test for R that is *not* a hardening test for the parent of R. More formally, Definition 14 defines a Weak Functionality Catching Test, $\mathsf{FCatching}_R(t)$.

Definition 14 (Weak Functionality Catching Test).

$$\begin{gathered} FCatching_R(t) \\ \Leftrightarrow \\ \neg(Hardening_{Parent(R)}(t)) \wedge Catching_R(t) \\ \Leftrightarrow \\ (\neg(Builds_{Parent(R)}(t)) \vee Fails_{Parent(R)}(t) \vee \\ \neg(\exists R'.Parent(R) = Parent(R) \wedge Fails_{R'}(t) \wedge Oracle_{R'}(t) = \texttt{fail})) \\ \wedge \\ Fails_R(t) \end{gathered}$$

The definition of a weak functionality catching test is simple and intuitive: it is one that is catching but not hardening. However, instantiating this simple definition, reveals several implicit criteria. The test might not build on the parent, or it could build yet fail on the parent. There is also the possibility that there does not exist a revision that shares the same parent and for which the test correctly fails on that revision. This last criterion arises due to the 'weak' nature of the catch.

If we know that the test *correctly fails* on the current revision (a 'strong' catch), then this awkward criterion disappears. This can be seen in the definition of strong functionality catching test. A *Strong Functionality Catching* test t, for revision R is a strong catching test for R that is *not* a hardening test for the parent of R. More formally, Definition 15 defines a Strong Functionality Catching Test, $\text{SFCatching}_R(t)$.

Definition 15 (Strong Functionality Catching Test).

$$\begin{gathered} SFCatching_R(t) \\ \Leftrightarrow \\ \neg(Hardening_{Parent(R)}(t)) \wedge SCatching_R(t) \\ \Leftrightarrow \\ (\neg(Builds_{Parent(R)}(t)) \vee Fails_{Parent(R)}(t)) \\ \wedge \\ Fails_R(t) \wedge Oracle_R(t) = \texttt{fail} \end{gathered}$$

As an example, suppose a pull request updates the `Product` object to introduce a method, `distance`, that computes the distance to each supplier. Suppose further that the pull request contains a bug that incorrectly omits an `abs` operator that is required to ensure that differences in location are always positive. Example 6 is a strong functionality catching test. That is, it will not build on the parent of the pull request (since it calls `distance`, which does not exist in the parent). It also fails correctly on the pull request, since the distance is incorrectly computed as a negative number.

Example 6 (Strong Functionality Catching Test).

```
def distance_test():
  test_supplier = MOCK_PRODUCT(Locations.LONDON)
  dist = test_supplier.distance()
  assertTrue(dist>=0)
```

Now, suppose that the author of the pull request fixes the bug as a result of the failure of the test in Example 6. Now Example 6 correctly passes on the fixed pull request. As such, the test now becomes a (strong) hardening test for the pull request revision, protecting it against future regressions in the `distance` method.

4 EIGHT CATEGORIES OF TEST

The 8 possible combinations of strong/weak hardening on parent and catching on child and their intersections are shown in the Venn diagram in Figure 1. In each of these eight regions there reside tests that exhibit interesting behaviours, five of which (the shaded regions) give rise to misleading test signals, while only three of which lead to reliable true signal. In the remainder of this section we explain the eight categories of test depicted in Figure 1.

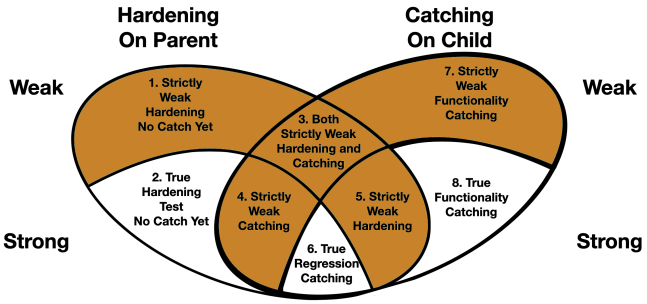


Figure 1: Hardening and catching Venn Diagram: Strong is a subset of weak. A test can be both (strong/weak) hardening on parent and also (strong/weak) catching on child. Each region of the Venn diagram corresponds to an interesting and important test behaviour.

Five misleading classes of signal (shaded regions in Figure 1): A test that is strictly weak hardening but not catching (Region 1) suggests that all is well, when it is not. A test that is strictly weak catching but not hardening (Region 7) may appear to catch new functionality but, because it is strictly weak catching, it is a 'fake' functionality catching test. The remaining three misleading signals include two combinations of strictly weak and strong hardening/catching (Regions 4 and 5), and finally, the most misleading of all (Region 3): both strictly weak hardening and strictly weak catching. A test that resides in this most misleading category might fail to catch a bug and subsequently 'complain' (i.e., incorrectly fail) when it is fixed.

Three classes of true signal: A test that is strong hardening, but not catching (Region 2), is simply a good hardening test that has yet to catch anything. A test that is strong catching but not hardening (Region 8) is a true functionality catching test. A strong catching test that is also a strong hardening (Region 6) is a true regression catching test.

Research Challenge 1: find techniques that can help reduce the risk of miss-classification of tests according to this classification.

5 DEPLOYMENT OF HARDENING AND CATCHING TESTS

In this section we review the options for deploying hardening and catching tests. In the case of timely hardening, this provides a way to formally analyse the (current widely-adopted) approach used in research and practice for automated regression test generation. For JiTTests, the analysis reveals the complexities and trade-offs in the decision processes, leading to an initial set of proposed deployment options. We do not claim these proposed deployment options are unequivocal nor that they are comprehensive but rather, that they illustrate how the formal analysis together with a case-by-case reasoning can shed light on the trade-offs involved.

5.1 Cases of Behaviour for Timely Hardening Tests

Table 1 captures[3] the possible combinations of outcomes of a test generated on a revision, which we call the parent, and that seeks to generate a hardening timely test for this revision. Such a test seeks to harden the parent against future regressions in a subsequent revision, which we call the child. In this mode of deployment, we generate tests on the parent pull request, where the test passes on this parent, and we have some unknown future children, for which we want to protect the code. From this case-by-case analysis, we can see that any test that passes on the current version of the system is worthy of consideration.

In order to distinguish between strictly weak and strong hardening tests, we can use the simple expedient of reporting the generated test to the engineer. Indeed, this is what we currently do with the ACH hardening tool [29]. It is also what many other other test generation tools do. Reporting the test to the engineer is not necessary: one could simply auto-land all passing generated tests into production, but such auto landing would be risky. It risks landing strictly weak hardening tests, or even tests that are *not* hardening at all because they can catch no future regression.

Actionable Conclusion: By seeking the engineer's review, we eliminate three (of the five) problematic categories (regions 1, 3 and 5 deplicted in Figure 1), thereby dramatically reducing potential future confusion from 'fake' test signals. Furthermore, the effort required from the engineer is low; they merely need to determine whether the assertion is reasonable, and therefore the test is right to pass on the current revision. This is a good example the principal that

> "A relatively small amount of timely oracle information can dramatically enhance test effectiveness."

In this case, a check that may take an engineer seconds to perform removes three fifths of the problematic test categories. Rather than thinking of the oracle as some unknowable theoretical construct, there is a more 'engineering orientated viewpoint': The oracle is not a discrete logical construct, but better thought of as a continuous resource on which we can draw. Since it is an *expensive* resource on which to draw, we need to make the engineering trade-off that determines when the cost is worth the benefit. The only situation where auto-landing generated tests might be safe from these problematic categories, is when tests are disposable. For example, where they are generated to detect any changed behaviour after some a specific meaning-preserving refactoring, and subsequently discarded.

Overall, therefore, we think it is safe to deploy hardening test generation and to land into production any tests that pass consistently (i.e., non flakily [20, 39, 57]) on the current revision, that are acceptable to human reviewers. While this can have problems, as indicated by the four problematic rows attributed to false negatives and false positives in Table 1, any such risks are merely specific instances of the risks that already pertain to human-written tests.

[3]The tables in this paper use a traffic light colour-coding scheme which is best viewed in colour, rather than in black-and-white. Green indicates a positive test experience, in which there is no friction from misleading signal and there is test benefit. Red denotes friction: the test does not catch bugs but rather slows down the engineering development. Amber indicates possible friction, that is deemed to be probably tolerable.

Deploying Hardening Timely Tests: Based on Table 1, when we generate a hardening test from a current revision, and the test passes, we will offer the test up for review by an engineer who may choose to land it into production. At the heart of this decision, resides the *core belief in the value of testing itself*:

> "The risk of future false positives is outweighed by the chance of catching future real bugs."

Research Challenge 2: automatically distinguish weak from strong hardening: find techniques that can take partial oracle information and use this to improve the chance that a weak hardening test is strong, to further reduce human oracle effort.

Research Challenge 3: automatically distinguish regression catching from false positive: find techniques that reduce the risk that a test will raise false positive signal, due to strictly weak catching.

5.2 Cases of Behaviour for JiTTests

We are specifically interested in the case of generated JiTTests, because a JiTTest is generated at the last occasion in which it is possible to catch a bug before it lands into production. This makes this particular testing paradigm highly important and impactful. Despite this importance, there has been very little discussion and analysis of the opportunities and challenges for generation of such JiTTests. Table 2 collects the possible outcomes for attempts to generate a JiTTest for a revision R, and the consequences of accepting the signal from the generated test.

For a JiTTest, the test generation tool has available to it, both the pull request, R, and its parent, $\mathsf{Parent}(R)$, at test generation time. The table groups the analysis results for the current revision and its parent into five categories, with a sub-table for each category. In this section, we consider each category and turn.

Category 1 (Parent Hardening): tests that pass on the parent. A hardening test may have been generated some months or even years before the first time it fails in production on a new revision. By contrast, a JiTTest is generated on the fly when a revision is submitted. Where the test passes on the parent revision, the JiTTest may be a hardening test for the parent, although it is not timely for the parent, only for the child, since it is a JiTTest.

Where the test passes on the parent, the case-by-case analysis is isomorphic to the case-by-case analysis for the hardening timely test. This is shown in the first eight rows of Table 2. Although isomorphic, the consequences of false positives are more problematic than for hardening timely tests, precisely because of the just-in-time nature of the test generation process.

Since the tests are generated just-in-time for the revision, it is likely that there is no existing test that fails. If there were, then we would not need to generate any further tests. Furthermore, there may simply be *no* existing tests at all (either passing or failing). This is precisely the scenario in which we would need to generate JiTTests, to either increase confidence in the revision under test or stop a bug form landing into production. For the same reason, when a JiTTest fails on a revision, it may be the only test that provides any signal, whether passing or failing. Therefore, the risk of high friction from a newly created false positive is considerably higher than for a long-standing hardening timely test.

Table 1: The possible outcomes for generated timely tests. The test is generated for the current revision (the parent). We distinguish the possible outcomes for the execution on some future child of the current revision. In describing the outcome for a child we are also describing the outcome for some future 'child of a child of a ...' for some arbitrary number of revisions for which the test passes. A test that fails on the child is one of the catching cases from Figure 1, whereas a test that passes on both the parent and child transitions from one hardening case (for parent) to another (for the child), hence the '→' in column 6.

outcome on		oracle for		type	In	
parent	child	parent	child	for child	Figure 1	Consequences if we were to accept the generated test's signal
When tests were landed to harden the parent and now they are executed on some future child ...						
pass	pass	pass	pass	true negative	2 → 2	Strong Hardening test passes: All is well; move fast
pass	pass	fail	pass	true negative	1 → 2	Strictly Weak Hardening test passes: the child fixes an uncaught bug
pass	fail	pass	fail	true positive	6	True regression catching: Catches the child's regression
pass	fail	fail	fail	true positive	5	Aparent regression is more likely to be a Failed error propagation [12]
pass	pass	fail	fail	false negative	1 → 1	Strictly Weak Hardening test continues to miss a bug
pass	pass	pass	fail	false negative	2 → 1	Strong Hardening test misses a bug introduced by the child
pass	fail	pass	pass	false positive	4	Strong Hardening test wrongly blocks the child
pass	fail	fail	pass	false positive	3	Strictly Weak Hardening test wrongly flagged a fix as a new failure
When tests were landed to harden the parent, but some future child makes them obsolete ...						
pass	no build	pass	—	—	2 → ⊥	Strong Hardening test retired: child removes tested functionality
pass	no build	fail	—	—	1 → ⊥	Strictly Weak Hardening test retired: Test was weak anyway

We may choose to suppress the signal of a JiTTest to avoid undue friction. The balance between risk and benefit will likely be determined differently by different organisations and different projects within an organisation. Notwithstanding these organizational preferences, a general principle applies to all: the risk of friction is higher for JiTTests than for merely timely tests.

Actionable Conclusion: As a result of this analysis, a failing JiTTest signal may be regarded as a 'trigger' to do further testing or analysis before troubling the engineer with the signal. By contrast passing JiTTest signal still plays the role of giving the engineer additional confidence, and such a passing JiTTest may also be a hardening JiTTest. It therefore makes sense to have a deployment route for JiTTests, if only to create hardening JiTTests as the side effect of attempting to generate catching JiTTests.

Category 2 (Child Hardening): tests that fail on the parent and pass on the child. For all four cases in Category 2, the JiTTest fails on the parent but passes on the current revision. In any such case, it is relatively safe to give the signal to the engineer. A Category 2 test that is a hardening JiTTest is, by definition, both hardening and timely for R, although it is neither hardening nor timely for its parent.

When the test is a true negative for the current revision, its behaviour not only generates a new hardening test for the future (as a hardening timely test would), it can yield additional signal value since it is just-in-time. For example, the test may show that the developer's change has fixed (or improved) the behaviour of the parent revision. Such fail → pass JiTTests therefore add value over and above purely generating new hardening timely tests (which they also do 'for free'). Also, fortunately, when the test is a false negative pass, the negative consequences are relatively low, as the case-by-case analysis reveals. This observation mimics the analysis for hardening timely tests in Section 5.1.

Actionable Conclusion: For both Categories 1 and 2 the generated test is a hardening timely test for the child but not for the parent. It is safe to give this timely hardening signal to the engineer when the test is generated for R.

Category 3 (New Functionality Tests): tests that fail to build on the parent. Where a JiTTest fails to build on the parent of the current revision and passes on the current revision, this is perfectly safe to land (as the case analysis in Table 2 shows). Such a test will become a new hardening test which, like others, may 'bake in' wrong functionality, but there is the hope that other tests will catch that. Furthermore, since the test fails to build on the parent, it is likely that this test is testing precisely the new functionality added by the revision. Once again, as with Category 2 (for tests that fail on the parent), the just-in-time nature of the test gives extra signal because it does not build on the parent.

Actionable Conclusion: From the analysis of Categories 2 and 3, we conclude that it is *always* wise to run a generated test on the parent of a revision (to obtain this extra signal). This is not something that current test tools tend to do, but our analysis suggests that it is not only worthwhile. Test generation might *specifically target* the generation of such tests, and check/report test behavior on the parent of the tested revision.

Category 4 (Cannot Land): tests that fail to build on the child. Where a JiTTest fails to build on the current revision, it clearly cannot be landed as a new test. Nevertheless, its signal may be useful to the engineer. For example, if it previously failed on the parent, then it may indicate a fix or an improved functionality in the current revision. The engineer may even choose to cite the generated test as *documentation* for the improvement that their revision achieves.

Actionable Conclusion: From the analysis of Category 4, we conclude that we cannot land tests that fail to build on the child, but it is always worth giving the signal to the engineer.

Category 5 (Likely Discard): tests that fail on both the parent and child. Finally, JiTTests that fail on both the parent and the current revision will typically have to be discarded. The only corner

Table 2: The possible outcomes for generation of Just-in-Time Tests (JiTTests). In this mode of deployment, tests are generated on the fly when a revision is first submitted. A test is generated when a revision R is submitted, based on the revision R, and its parent, Parent(R). The outcome of test execution on both the revision and the parent can be taken into account when determining whether to land the test, report its signal (but not land the test), or to discard it completely. In Column 6, JiTTests that fail on R correspond to one of the categoiries from Figure 1, for which a decision must be made about whther to give the signal to the engineer. Cases that do not fail on R may correspond to transitions between categories (Parent(R) $\rightarrow R$).

outcome on Parent(R)	outcome on R	oracle for Parent(R)	oracle for R	type for R	In Figure 1	Consequences of landing the JiTTest and reporting its signal
Category 1 (Parent Hardening): tests that pass on the parent ...						
pass	pass	pass	pass	true negative	$2 \rightarrow 2$	Strong Hardening JiTTest hardens R
pass	pass	fail	pass	true negative	$1 \rightarrow 2$	Strictly Weak Hardening on parent (missed bug likely fixed)
pass	fail	pass	fail	true positive	6	True Regression catching JiTTest
pass	fail	fail	fail	true positive	5	Strictly Weak Hardening JiTTest
pass	pass	fail	fail	false negative	$1 \rightarrow 1$	Strictly Weak Hardening JiTTest misses a long-standing issue
pass	pass	pass	fail	false negative	$2 \rightarrow 1$	Strong Hardening JiTTest missed a bug introduced by R
pass	fail	pass	pass	false positive	4	Strictly Weak Catching JiTTest wrongly blocks change
pass	fail	fail	pass	false positive	3	Doubly Weak JiTTest wrongly blocks a fix
Category 2 (Child Hardening): tests that fail on the parent and pass on the child ...						
fail	pass	pass	pass	true negative	$7 \rightarrow 2$	Strong Hardening JiTTest: R fixes a broken test
fail	pass	fail	pass	true negative	$8 \rightarrow 2$	Strong Hardening JiTTest correctly identifies a fix
fail	pass	pass	fail	false negative	$7 \rightarrow 1$	Strictly Weak Hardening JiTTest lets through new bug
fail	pass	fail	fail	false negative	$8 \rightarrow 1$	Strictly Weak Hardening JiTTest lets an existing bug remain
Category 3 (New Functionality Tests): tests that fail to build on the parent ...						
no build	pass	—	pass	true negative	$\perp \rightarrow 2$	Strong Hardening for new functionality introduced by R
no build	fail	—	fail	true positive	8	Strong Hardening/Catches R's new functionality bug
no build	pass	—	fail	false negative	$\perp \rightarrow 1$	Strictly Weak Hardening bakes in incorrect new functionality
no build	fail	—	pass	false positive	7	Strictly Weak Catching blocks a perfectly good change
Category 4 (Cannot Land): tests that fail to build on the child ...						
outcome on Parent(R)	outcome on R	oracle for Parent(R)	oracle for R	type for R		Consequences of reporting the JiTTest signal to the engineer
pass	no build	pass	—	—	$2 \rightarrow \perp$	Strong Hardening but tested functionality is removed by R
fail	no build	fail	—	—	$8 \rightarrow \perp$	Strong Catching JiTTest may flag the bug removed by R
fail	no build	pass	—	—	$7 \rightarrow \perp$	Strictly Weak Catching false positive, but it is removed
pass	no build	fail	—	—	$1 \rightarrow \perp$	Strictly Weak Hardening weakness means little valuable signal
Category 5 (Likely Discard): tests that fail on both the parent and child ...						
outcome on Parent(R)	outcome on R	oracle for Parent(R)	oracle for R	type for R		Consequences of discarding the JiTTest and ignoring its signal
fail	fail	pass	pass	false positive	7	Strictly Weak Catching JiTTest should be discarded
fail	fail	fail	pass	false positive	7	Strictly Weak Catching so would have rejecting a correct fix
fail	fail	pass	fail	true positive	8	Discarding Strong Catching JiTTest meant missing a new bug
fail	fail	fail	fail	true positive	8	Discarding Strong Catching means missing existing bug

case is one where we believe we have discovered a long-standing or new bug, but there will need to be an additional way to gain confidence that this is what the test is revealing rather than a false positive.

The risk of friction from a false positive is relatively high. The chances of finding such a bug with a test that fails on the parent and the current revision is low. Therefore, it seems likely that the balance probabilities is in favour of discarding all such tests.

Actionable Conclusion: From the analysis of Category 5, without new techniques, we may sadly have to discard tests that fail on both the parent and the child. This is the least certain of our actionable conclusions from the analysis in this section. Futhure research will hopefully find ways to distinguish sub-categories; one aspect of the Catching JiTTest challenge (see Section 6).

Deploying catching tests: We summarize the foregoing actionable conclusions to document the eight possible outcomes that can be observed, irrespective of the oracle in Table 3.

6 RESEARCH CHALLENGE 4: THE CATCHING JITTEST CHALLENGE

Earlier in the paper, we discussed Research Challenges 1 – 3, which concerned classification and hardening tests. These three challenges

Table 3: Suggested JiTTest deployment approach based on signal from parent and current revision. Based on the case by case analysis in Table 2 we give a proposed deployment approach for each case. Of course, unlike the analysis from Table 2, oracle information is assumed to be unavailable, and therefore we do not know whether signal is a true/false positive/negative.

Category 1: When JiTTest passes on parent of current revisions ...				
outcome on Parent(R)	R	Decision	Possible undesirable consequences	Best case consequences
pass	pass	Land Test	Bakes in long standing issue	Catches future regressions
pass	fail	Report Fail	False positive; but other tests may pass	Catches a current regression
Category 2: When JiTTest fails on parent but passes on current revision ...				
outcome on Parent(R)	R	Decision	Worst case consequences of landing new test	Best case consequences of landing new test
fail	pass	Land Test	Bakes in newly created issue	Catches future regressions in new functionality
Category 3: When JiTTest does not build on the parent of the current revision ...				
outcome on Parent(R)	R	Decision	Worst case consequences	Best case consequences
no build	pass	Land Test	Bakes in newly created issue	Catches future regressions in new functionality
no build	fail	Report Fail	False positive; but other tests may pass	Catches bug in new functionality
Category 4: When JiTTest cannot land but may still give useful signal on the revision ...				
outcome on Parent(R)	R		Worst case consequences of reporting signal	Best case consequences of reporting signal
pass	no build	Give Signal	Wrongly claims that parent was correct	Confirms that behaviour was removed
fail	no build	Give Signal	Wrongly claims that parent was incorrect	Helps to document a fix
Category 5: When JiTTest fails on the current revision and its parent ...				
outcome on Parent(R)	R		Worst case consequences of discarding JiTTest	Best case consequences of discarding JiTTest
fail	fail	Discard Test	Loses a bug catch on current revision	Loses a potentially misleading test

are important, but the most challenging of all (and likely the most impactful) is Research Challenge 4: The Catching JiTTest Challenge, which we summarise as follows:

> "Given a buggy pull request and its parent, automatically generate a test that can reveal bugs in the pull request, with low risk of false positive failure on non-buggy pull requests. "

We call this the 'Catching JiTTest Challenge'. It is essentially to improve the value of JiTTest R@P=p (See Section 3.5), for some suitably high choice of precision threshold p, such as 0.8.

Suppose a JiTTest, t, generated just-in-time for a revision, R, passes on R and also happens to pass on Parent(R). This test, t, is simply a hardening JiTTest (for R). However, if t fails (or fails to build) on Parent(R), then t is a specific new kind of test case that can be found *only* using the Just-in-Time paradigm: It could not be landed at any time before R has landed, because it would land a failing test signal or even break the build.

The Catching JiTTest Challenge thus opens up a whole new class of test cases that can be generated. In order to tackle the challenge, we are inherently faced with a particularly pernicious example of the Oracle Problem: the only oracle information may be that in the pull request itself. Such information can be exceptionally parsimonious.

6.1 The Parsimonious Pull Request Problem and the Oracle Scavenging Solution

As an extreme example of a parsimonious pull request, consider the following hypothesised (but entirely plausible) pull request:

title: add location details to lookup query
summary: TSIA
Test Plan: YOLO

Let us call this pull request *TSIA*. The acronym 'TSIA' stands for 'Title Says It All', while the acronym 'YOLO' stands for 'You Only Live Once'. Engineers may believe this level of detail is sufficient for the reviewer. Therefore, it is also the only information available to any automatic technology seeking to design tests.

Code Summarization May Prove Insufficient: Of course, we could seek to use LLMs for code summarization [86] to provide a better description in the summary. There has been much research work on this problem [26], and it would undoubtedly help the *human* reviewer. Nevertheless, when our goal is to automatically provide a better test plan than 'YOLO', we would need to be cautious to ensure that our LLM-based test generation techniques do not become victims to the obvious circularity involved by also using LLMs to infer the code summaries.

Grounds for Optimism: Plenty of Computation Time: As revealed in Section 2.4, the timing constraints for online test generation are considerably more relaxed than might, at first, be thought.

In deployment to continuous integration we are typically afforded approximately eight hours to generate JiTTests. This allows for considerable static and dynamic analysis, and other automated techniques to be deployed to determine what signal, if any, should be given to the pull request reviewer.

Grounds for Optimism: Tackling the Oracle with LLMs: In the remainder of this section, we give two examples that illustrate ways in which, even with a highly parsimonious pull request, we are able to glean sufficient information to weed out false positives in both putative regression catching and functionality catching JiTTests.

6.2 Weeding out Fake Regression Catching

Suppose we have a hardening JiTTest, *RCJ*, that we are considering as a potential Regression Catching JiTTest.

Since we consider it to be a regression catching test, then, by Definition 12, *RCJ* passes on Parent(*TSIA*) and fails on *TSIA*. Now suppose that *RCJ* asserts that the result of a query in the executed code returns a `NULL` field for the location column. Such a test is seeking to *enforce* precisely the behaviour that we might reasonably infer *TSIA* is seeking to extend/change. If we present this test to the pull request author as a 'catch' we are likely to significantly increase the friction in their development experience. They will be understandably frustrated that their pull request, title 'said it all', and our generated test paid little attention to this.

Even with this extremely parsimonious pull request, there is enough information in the title alone to automatically determine that *RCJ should* be discarded, and its signal *not* passed to the engineer (unless it is to confirm that they have changed the functionality in the way they intended). Therefore, an automated technique *would* be able to identify that the *RCJ* test failure is likely to be a false positive, even with only those six words of the *TSIA* pull request title and the behaviour of *RCJ* as guidance.

6.3 Weeding out Fake Functionality Catching

Suppose the code that implements the parsimonious pull request *TSIA* includes logic that calculates the fuel consumed per distance travelled, as a report on fuel efficiency. Suppose that this logic is computed in terms of the location, and that the code in the pull request, relevant to this computation, is as follows.

```
# Fuel efficiency computation assumes the destination
# location cannot be the same as the current location
distance = distance(current_location, destination)
fuel_efficiency = fuel_consumed/distance
```

Suppose further that the pull request also includes the implementation of the new function `distance`.

Now suppose that we automatically generate a test *FCJ* that, on the face of it, appears to be a functionality catching JiTTest. By Definition 14, this means that *FCJ* fails to build (or builds yet fails) on the parent, and also fails on the pull request. The new test, *FCJ*, calls the new logic to determine the distance and, therefore, does not build on the parent. So far, so good: the test appears to be a functionality catching JiTTest.

Now comes the challenge: in order to be sure that the failure on *TSIA* is a true positive, we need to scavenge as much oracle information as possible. Suppose *FCJ* fails with a divide by zero exception in the computation of `fuel_efficiency`, and that it does this because the current and destination locations turn out to be identical in the generated test case. In this situation, the comment that immediately proceeds the computation of distance in the pull request is a highly relevant clue. An automated test technique, especially one with the power of Large Language Model, *ought* to be able to infer, from this clue, that *FCJ* is failing precisely because it does not respect the assumption captured by this comment.

Notice that this does not mean that we should not report *any* signal to the engineer. Rather, it would be better to construct a comment on the pull request that observes that the test fails because the assumption is not met.

Test signal refinement → Test Improvement: Furthermore, the tool could transform the test into a *passing* test that documents the assumption by asserting that the distance computation will raise a divide-by-zero exception when the current and destination location are identical. This might encourage the engineer to make an improvement that avoids an exception.

Test Improvement → Automated Repair: The automated system could also *suggest* such an improvement: test generation thereby leading to improvement/repair suggestion.

LLM + non-executable text = Oracle: As these two simple examples illustrate, the Catching JiTTest challenge is demanding, but not insurmountably so. In particular, we believe that LLMs' ability to reason with both code and natural language (in the same inference) provides a perfect bridge between the code and any available oracle information in non-executable text such as comments and other documentation, parsimonious though it may be. We see grounds for optimism in the use of '*oracle scavenging*'; the hunt for non-executable code hints to expected behaviour. There has been much recent work on LLMs for testing [26, 82], including recent work on the oracle inference problem [27, 47, 54, 64], but their potential to combine with test generation to tackle the Catching JiTTest Challenge remains currently open and untackled, yet highly impactful.

6.4 Oracle Scavenging for Non-executable Code

Non-executable text is primarily authored by humans, and it will increasingly become the way in which humans drizzle oracle information into software. This human author is arguably the best author for such documentation, because human authors can provide these crucial fragments of oracle information. As discussed in Section 3.2, Software Engineering is an inherently *social* process. The oracle is one of the key points at which the social nature of the engineering discipline impinges. Different humans may even disagree about what the expected behaviour of the software system should be.

The oracle may change over time, even with the same engineer changing their view of expected behaviour over time. Regulatory and other changes may also impact the expected behaviour of software. Therefore, without *any* further change in the code, the oracle may change. It will become increasingly important, as language models play an increasingly wide role in code generation, to efficiently and accurately capture oracle information.

As the charter for the Source Code Analysis and Manipulation workshop points out:

> "Source code contains the only precise description of the behaviour of the system" [10].

This makes source code analysis and manipulation extremely important, and it always will [34], but it also highlights what source code *cannot* do, as well as what it can do. It is the only precise description of behaviour the system, but it is only that: a description of the behaviour of the system, not a description of its intended behaviour. For the purpose of oracle extraction, we have to treat executable text (the executable source code) as potentially suspicious, and focus on extraction of intended semantics from non-executable text, so that we can compare intended and actual semantics. This non-executable text may also be suspicious, of course, but at least it provides an alternative view of expected behaviour. In the near future, there will be equal/greater importance for non-executable system descriptions, so programming languages will need to evolve to support this.

Perhaps we may even see a resurgence of interest in literate programming [53]. In literate programming, the aim of non-executable text was to describe *how* the executable code performs its task. Programming languages will need to evolve further to also include at least some informal specification of *what* those tasks are; what is the intended behaviour of the system. We find ourselves back in previously well-trodden territory: agonizing over the absence of specifications and the relationship between specification and testing. Formal specifications can help with test generation [43], but the kind of oracle information/specification required need not be formal. Furthermore, the uncertainty about the true intended behaviour suggests that it would not even be desirable to have to rely on a purely formal specification.

6.5 JiTTests Can Also Find Latent Bugs

Many organisations have large legacy code bases and systems that contain latent bugs that have resided in the code base, sometimes for years. By definition, such latent bugs are not mission-critical, since they have survived some time without correction. Nevertheless, although not mission-critical, they may, collectively, pose significant problems for users and can also unnecessarily drain resources and computational capacity.

Furthermore, some of these latent bugs may currently benefit from failed error propagation [12]. In this failed error propagation situation, the bug is executed and 'infects' the local state of computation, but this infection never manifests itself as a failure, because the bug is suppressed along all paths between the infected state and some point at which it becomes observable. Such failed error propagation scenarios are vulnerable to future pull requests, that may inadvertently unblock a path from the infection point to an observation point, whereupon the latent bug newly leads to a failure; possibly a mission-critical failure. In this regard, such latent bugs which currently benefit from failed error propagation are the software equivalent of unexploded bombs in the code base, they simply await detonation.

A Catching JiTTest is 'just-in-time' for the pull request that it tests. It might therefore appear that, by definition, it cannot have a role to play in catching latent bugs, but this is not the case. We can take an arbitrary section of code of interest, delete it, and then re-insert it as a new pull request. For example, suppose we pick a particular method, m. We delete m and subsequently re-insert it as a new pull request. Suppose a Functionality Catching JiTTest, t, is generated for the pull request. The test t will thereby reveal a latent bug in m, that has already landed into production.

Oracle Scavenging: The challenge of finding latent bugs with JiTTests lies primarily in locating additional sources of oracle information that help to determine what a re-inserted method (or code fragment) m is *supposed* to do. When we seek to find latent bugs we have the most parsimonious pull request of all, from which we can scavenge no new non-executable text. Fortunately, even for this most challenging form of oracle scavenging, we can still automatically scour the continuous integration history for relevant past pull requests, extracting previous non-executable text from them. We can also seek out other documentation in the form of comments and other sources of non-executable text. Indeed, this new world of LLM-based code generation (and test generation) dramatically increases the importance of such non-executable text: it is no longer just written for human consumption, it must also be written for machine consumption.

No matter what way we seek to incorporate oracle information into the test generation approach, we will be able to use JiTTest generation as an approach to uncover latent faults in existing software systems. We can simply apply the delete-and-re-insert process arbitrarily many times to walk over a code base, using JiTTest generation to uncover latent bugs.

7 CONCLUSIONS AND FUTURE WORK

This paper explores the complex interplay between test generation, test deployment and test oracles. Previous work has tended to focus solely on test generation, and the regression oracle. We show that modes of test deployment have a profound impact on the way we seek to automatically generate tests and report their signal to engineers. Our analysis also reveals the impact of taking non-regression oracle fragments into account.

Our primary contribution is to formally define and crystallise the research challenges that emerge from this three-way interplay. We hope that the paper clarifies the impact that this research agenda can have on industrial software testing. Our aim is to stimulate the research community to take up and investigate the challenges raised herein, underpinned by precise formal definitions of the challenges involved, and motivated by industrial experience that highlights their potential impact.

ACKNOWLEDGEMENTS

We would like to thank the leadership of Meta's Product Compliance and Privacy, Fundamental Artificial Intelligence Research (FAIR), Developer Infrastructure (DevInfra), and Instagram Product Foundation teams for supporting our work over the past decade, and the many Meta software engineers and testers whose experience, expertise, and engagement have helped shape the ideas presented here. We would also like to thank the many academics and other researchers with whom we have had the enormous pleasure to interact over 3+ decades of research work on verification, testing and AI.

REFERENCES

[1] John Ahlgren, Maria Eugenia Berezin, Kinga Bojarczuk, Elena Dulskyte, Inna Dvortsova, Johann George, Natalija Gucevska, Mark Harman, Ralf Laemmel, Erik Meijer, Silvia Sapora, and Justin Spahr-Summers. 2020. WES: Agent-based User Interaction Simulation on Real Infrastructure (keynote paper). In 8th *Genetic improvement workshop (GI at ICSE 2020)*, Shin Yoo, Justyna Petke, Westley Weimer, and Bobby R. Bruce (Eds.). ACM, 276–284.

[2] John Ahlgren, Maria Eugenia Berezin, Kinga Bojarczuk, Elena Dulskyte, Inna Dvortsova, Johann George, Natalija Gucevska, Mark Harman, Maria Lomeli, Erik Meijer, Silvia Sapora, and Justin Spahr-Summers. 2021. Testing Web Enabled Simulation at Scale Using Metamorphic Testing. In *International Conference on Software Engineering (ICSE) Software Engineering in Practice (SEIP) track*. Virtual.

[3] John Ahlgren, Kinga Bojarczuk, Sophia Drossopoulou, Inna Dvortsova, Johann George, Natalija Gucevska, Mark Harman, Maria Lomeli, Simon Lucas, Erik Meijer, Steve Omohundro, Rubmary Rojas, Silvia Sapora, Jie M. Zhang, and Norm Zhou. 2021. Facebook's Cyber–Cyber and Cyber–Physical Digital Twins (keynote paper). In *25th International Conference on Evaluation and Assessment in Software Engineering (EASE 2021)*. Virtual.

[4] Nadia Alshahwan, Arianna Blasi, Kinga Bojarczuk, Andrea Ciancone, Natalija Gucevska, Mark Harman, Michal Krolikowski, Rubmary Rojas, Dragos Martac, Simon Schellaert, et al. 2024. Enhancing Testing at Meta with Rich-State Simulated Populations. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*. 1–12.

[5] Nadia Alshahwan, Jubin Chheda, Anastasia Finegenova, Mark Harman, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Automated unit test improvement using Large Language Models at Meta. In *ACM International Conference on the Foundations of Software Engineering (FSE 2024)* (Porto de Galinhas, Brazil, Brazil).

[6] Nadia Alshahwan, Xinbo Gao, Mark Harman, Yue Jia, Ke Mao, Alexander Mols, Taijin Tei, and Ilya Zorin. 2018. Deploying Search Based Software Engineering with Sapienz at Facebook (keynote paper). In 10th *International Symposium on Search Based Software Engineering (SSBSE 2018)*. Montpellier, France, 3–45. Springer LNCS 11036.

[7] Nadia Alshahwan, Mark Harman, and Alexandru Marginean. 2023. Software Testing Research Challenges: An Industrial Perspective (keynote paper). In *2023 IEEE Conference on Software Testing, Verification and Validation (ICST 2023)*. IEEE, 1–10.

[8] Nadia Alshahwan, Mark Harman, Alexandru Marginean, Shubho Sengupta, and Eddy Wang. 2024. Assured LLM-Based Software Engineering (keynote paper). In $2^{nd.}$ *ICSE workshop on Interoperability and Robustness of Neural Software Engineering (InteNSE)* (Lisbon, Portugal).

[9] Nadia Alshahwan, Mark Harman, Alexandru Marginean, and Eddy Wang. 2024. Observation-based unit test generation at Meta. In *Foundations of Software Engineering (FSE 2024)*.

[10] Source Code Analysis and Manipulation Workshop (SCAM). 2001. SCAM Charter. https://www.ieee-scam.org/ The cited quotation has remained part of the workshop's credo since its foundation in 2001.

[11] Saswat Anand, Antonia Bertolino, Edmund Burke, Tsong Yueh Chen, John Clark, Myra B. Cohen, Wolfgang Grieskamp, Mark Harman, Mary Jean Harrold, Jenny Li, Phil McMinn, and Hong Zhu. 2013. An orchestrated survey of methodologies for automated software test case generation. *Journal of Systems and Software* 86, 8 (August 2013), 1978–2001.

[12] Kelly Androutsopoulos, David Clark, Haitao Dan, Mark Harman, and Robert Hierons. 2014. An Analysis of the Relationship between Conditional Entropy and Failed Error Propagation in Software Testing. In 36th *International Conference on Software Engineering (ICSE 2014)*. Hyderabad, India, 573–583.

[13] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. The Oracle Problem in Software Testing: A Survey. *IEEE Transactions on Software Engineering* 41, 5 (May 2015), 507–525.

[14] Carolin Brandt and Andy Zaidman. 2022. Developer-centric test amplification: The interplay between automatic generation human exploration. *Empirical Software Engineering* 27, 4 (2022), 96.

[15] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. Klee: unassisted and automatic generation of high-coverage tests for complex systems programs.. In *OSDI*, Vol. 8. 209–224.

[16] Cristian Cadar, Patrice Godefroid, Sarfraz Khurshid, Corina S. Păsăreanu, Koushik Sen, Nikolai Tillmann, and Willem Visser. 2011. Symbolic execution for software testing in practice: preliminary assessment. In 33rd *International Conference on Software Engineering (ICSE'11)* (Waikiki, Honolulu, HI, USA). ACM, New York, NY, USA, 1066–1071.

[17] Cristian Cadar and Koushik Sen. 2013. Symbolic Execution for Software Testing: Three Decades Later. *Commun. ACM* 56, 2 (Feb. 2013), 82–90.

[18] C. Calcagno, D. Distefano, J. Dubreil, D. Gabi, P. Hooimeijer, M. Luca, P. W. O'Hearn, I. Papakonstantinou, J. Purbrick, and D. Rodriguez. 2015. Moving Fast with Software Verification. In *NASA Formal Methods - 7th International Symposium*. 3–11.

[19] Thierry Titcheu Chekam, Mike Papadakis, Yves Le Traon, and Mark Harman. 2017. An empirical study on mutation, statement and branch coverage fault revelation that avoids the unreliable clean program assumption. In *Proceedings of the 39th International Conference on Software Engineering, ICSE 2017, Buenos Aires, Argentina, May 20-28, 2017*. 597–608.

[20] Maxime Cordy, Renaud Rwemalika, Adriano Franci, Mike Papadakis, and Mark Harman. 2022. FlakiMe: Laboratory-Controlled Test Flakiness Impact Assessment. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 982–994.

[21] Richard A. De Millo, Richard J. Lipton, and Alan J. Perlis. 1979. Social Processes and Proofs of Theorems and Programs. *Commun. ACM* 22, 5 (May 1979), 271–280. An earlier version appeared in *ACM Symposium on Principles of Programming Languages (POPL) , Los Angeles, California* , 1977 pp. 206–214.

[22] Edsger W. Dijkstra. 1978. On a Political Pamphlet from the Middle Ages (A response to the paper 'Social Processes and Proofs of Theorems and Programs' by DeMillo, Lipton, and Perlis). *ACM SIGSOFT, Software Engineering Notes* 3, 2 (1978), 14–17.

[23] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W O'Hearn. 2019. Scaling static analyses at Facebook. *Commun. ACM* 62, 8 (2019), 62–70.

[24] Rajendra T Dodhiawala, NS Sridharan, Peter Raulefs, and Cynthia Pickering. 1989. Real-Time AI Systems: A Definition and An Architecture.. In *IJCAI*. Citeseer, 256–264.

[25] Sebastian G. Elbaum, Hui Nee Chin, Matthew B. Dwyer, and Matthew Jorde. 2009. Carving and Replaying Differential Unit Test Cases from System Test Cases. *IEEE Transactions on Software Engineering* 35, 1 (2009), 29–45.

[26] Angela Fan, Beliz Gokkaya, Mitya Lyubarskiy, Mark Harman, Shubho Sengupta, Shin Yoo, and Jie Zhang. 2023. Large Language Models for Software Engineering: Survey and Open Problems. In *ICSE Future of Software Engineering (FoSE 2023)*.

[27] Zhiyu Fan, Haifeng Ruan, Sergey Mechtaev, and Abhik Roychoudhury. 2024. Oracle-guided Program Selection from Large Language Models. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 628–640.

[28] Jean Claude Fernandez, Claude Jard, Thierry Jéron, and César Viho. 1996. Using on-the-fly verification techniques for the generation of test suites. In *Computer Aided Verification: 8th International Conference, CAV'96 New Brunswick, NJ, USA, July 31–August 3, 1996 Proceedings 8*. Springer, 348–359.

[29] Christopher Foster, Abhishek Gulati, Mark Harman, Inna Harper, Ke Mao, Jillian Ritchey, Hervé Robert, and Shubho Sengupta. 2025. Mutation-Guided LLM-based Test Generation at Meta. In *2025 ACM Conference on Foundations of Software Engineering (FSE 2025)*. ACM. Also available as arXiv preprint arXiv:2501.12862.

[30] Phyllis G. Frankl, Stewart N. Weiss, and Cang Hu. 1997. All-Uses vs Mutation Testing: An Experimental Comparison of Effectiveness. *Journal of Systems Software* 38 (1997), 235–253.

[31] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: automatic test suite generation for object-oriented software. In 8th *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE '11)*. ACM, 416–419.

[32] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: directed automated random testing. In *Programming Language Design and Implementation (PLDI 2005)*, Vivek Sarkar and Mary W. Hall (Eds.). ACM, 213–223.

[33] Rahul Gopinath, Carlos Jensen, and Alex Groce. 2014. Code coverage for suite evaluation by developers. In *Proceedings of the 36th international conference on software engineering*. 72–82.

[34] Mark Harman. 2010. Why Source Code Analysis and Manipulation Will Always Be Important (Keynote Paper). In 10th *IEEE International Working Conference on Source Code Analysis and Manipulation*. Timisoara, Romania.

[35] Mark Harman, Yue Jia, and William B. Langdon. 2011. Strong Higher Order Mutation-Based Test Data Generation. In 8th *European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE '11)* (Szeged, Hungary). ACM, New York, NY, USA, 212–222.

[36] Mark Harman, Yue Jia, and Yuanyuan Zhang. 2015. Achievements, open problems and challenges for search based software testing (keynote Paper). In 8th *IEEE International Conference on Software Testing, Verification and Validation (ICST 2015)*. Graz, Austria.

[37] Mark Harman and Bryan F. Jones. 2001. Search Based Software Engineering. *Information and Software Technology* 43, 14 (Dec. 2001), 833–839.

[38] Mark Harman, Afshin Mansouri, and Yuanyuan Zhang. 2012. Search Based Software Engineering: Trends, Techniques and Applications. *Comput. Surveys* 45, 1 (November 2012), 11:1–11:61.

[39] Mark Harman and Peter O'Hearn. 2018. From Start-ups to Scale-ups: Opportunities and Open Problems for Static and Dynamic Program Analysis (keynote paper). In 18th *IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM 2018)*. Madrid, Spain, 1–23.

[40] Mark Harman and Laurence Tratt. 2007. Pareto Optimal Search-based Refactoring at the Design Level. In 9th *annual conference on Genetic and evolutionary computation (GECCO 2007)*. ACM Press, London, UK, 1106 – 1113.

[41] Mark Harman, Xiangjuan Yao, and Yue Jia. 2014. A Study of Equivalent and Stubborn Mutation Operators Using Human Analysis of Equivalence. In 36th *International Conference on Software Engineering (ICSE 2014)*. Hyderabad, India, 919–930.

[42] Kazi Amit Hasan, Marcos Macedo, Yuan Tian, Bram Adams, and Steven Ding. 2023. Understanding the time to first response in GitHub pull requests. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*. IEEE, 1–11.

[43] Rob Hierons, Kirill Bogdanov, Jonathan Bowen, Rance Cleaveland, John Derrick, Jeremy Dick, Marian Gheorghe, Mark Harman, Kalpesh Kapoor, Paul Krause, Gerald Luettgen, Tony Simons, Sergiy Vilkomir, Martin Woodward, and Hussein Zedan. 2009. Using Formal Methods to Support Testing. *Comput. Surveys* 41, 2 (Feb. 2009). Article 9.

[44] Charles Anthony Richard Hoare. 1996. How did software get so reliable without proof?. In *FME '96: Industrial Benefit and Advances in Formal Methods: Third International Symposium of Formal Methods Europe (LNCS, 1051)*. Springer-Verlag, 1–17.

[45] Charles Anthony Richard Hoare. 1996. How did software get so reliable without proof?. In *IEEE International Conference on Software Engineering (ICSE'96)*. IEEE Computer Society Press, Los Alamitos, California, USA. Keynote talk and extended abstract.

[46] Steven CH Hoi, Doyen Sahoo, Jing Lu, and Peilin Zhao. 2021. Online learning: A comprehensive survey. *Neurocomputing* 459 (2021), 249–289.

[47] Soneya Binta Hossain and Matthew Dwyer. 2024. TOGLL: Correct and strong test oracle generation with LLMs. *arXiv preprint arXiv:2405.03786* (2024).

[48] Eugene Ie, Chih-wei Hsu, Martin Mladenov, Vihan Jain, Sanmit Narvekar, Jing Wang, Rui Wu, and Craig Boutilier. 2019. RecSim: A Configurable Simulation Platform for Recommender Systems. *arXiv e-prints*, Article arXiv:1909.04847 (Sep 2019). arXiv:1909.04847 [cs.LG]

[49] Laura Inozemtseva and Reid Holmes. 2014. Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th international conference on software engineering*. 435–445.

[50] Kush Jain, Gabriel Synnaeve, and Baptiste Rozière. 2024. TestGenEval: A real world unit test generation and test completion benchmark. *arXiv preprint arXiv:2410.00752* (2024).

[51] Yue Jia and Mark Harman. 2011. An Analysis and Survey of the Development of Mutation Testing. *IEEE Transactions on Software Engineering* 37, 5 (September–October 2011), 649 – 678.

[52] René Just, Darioush Jalali, Laura Inozemtseva, Michael D. Ernst, Reid Holmes, and Gordon Fraser. 2014. *Are Mutants a Valid Substitute for Real Faults in Software Testing?* Technical Report UW-CSE-14-02-02. University of Washington.

[53] Donald E. Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (1984), 97–111.

[54] Michael Konstantinou, Renzo Degiovanni, and Mike Papadakis. 2024. Do LLMs generate test oracles that capture the actual or the expected program behaviour? *arXiv preprint arXiv:2410.21136* (2024).

[55] Hermann Kopetz and Wilfried Steiner. 2022. Real-Time Communication. In *Real-time systems: Design principles for distributed embedded applications*. Springer, 177–200.

[56] Kiran Lakhotia, Mark Harman, and Hamilton Gross. 2013. AUSTIN: An Open Source Tool for Search Based Software Testing of C Programs. *Journal of Information and Software Technology* 55, 1 (January 2013), 112–125.

[57] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An empirical analysis of flaky tests. In 22nd *International Symposium on Foundations of Software Engineering (FSE 2014)*, Shing-Chi Cheung, Alessandro Orso, and Margaret-Anne Storey (Eds.). ACM, Hong Kong, China, 643–653.

[58] Lech Madeyski, Wojciech Orzeszyna, Richard Torkar, and Mariusz Jozala. 2013. Overcoming the equivalent mutant problem: A systematic literature review and a comparative experiment of second order mutation. *IEEE Transactions on Software Engineering* 40, 1 (2013), 23–42.

[59] Valentin J. M. Manès, HyungSeok Han, Choongwoo Han, Sang Kil Cha, Manuel Egele, Edward J. Schwartz, and Maverick Woo. 2018. The Art, Science, and Engineering of Fuzzing: A Survey. *CoRR* abs/1812.00140 (2018). arXiv:1812.00140

[60] Ke Mao, Mark Harman, and Yue Jia. 2016. Sapienz: Multi-objective Automated Testing for Android Applications. In *International Symposium on Software Testing and Analysis (ISSTA 2016)*. 94–105.

[61] Ke Mao, Timotej Kapus, Lambros Petrou, Ákos Hajdu, Matteo Marescotti, Andreas Löscher, Mark Harman, and Dino Distefano. 2022. FAUSTA: Scaling Dynamic Analysis with Traffic Generation at WhatsApp. In *15th IEEE Conference on Software Testing, Verification and Validation, ICST 2022, Valencia, Spain, April 4-14, 2022*. IEEE, 267–278.

[62] James McGill. 2025. Time to First Review. https://docs.velocity.codeclimate.com/en/articles/2913584-time-to-first-review

[63] Phil McMinn. 2004. Search-based Software Test Data Generation: A Survey. *Software Testing, Verification and Reliability* 14, 2 (June 2004), 105–156.

[64] Facundo Molina, Alessandra Gorla, and Marcelo d'Amorim. 2024. Test Oracle Automation in the era of LLMs. *ACM Transactions on Software Engineering and Methodology* (2024).

[65] Morten Mossige, Arnaud Gotlieb, and Hein Meling. 2015. Testing robot controllers using constraint programming and continuous integration. *Information and Software Technology* 57 (2015), 169–185.

[66] Vijayaraghavan Murali, Chandra Maddila, Imad Ahmad, Michael Bolin, Daniel Cheng, Negar Ghorbani, Renuka Fernandez, Nachiappan Nagappan, and Peter C Rigby. 2024. CodeCompose: A Large-Scale Industrial Deployment of AI-assisted Code Authoring. In *Foundations of Software Engineering (FSE 2024)*. Porto de Galinhas, Brazil. Earlier version available as arXiv:2305.12050.

[67] Peter W O'Hearn. 2019. Incorrectness logic. *Proceedings of the ACM on Programming Languages* 4, POPL (2019), 1–32.

[68] Mike Papadakis, Yue Jia, Mark Harman, and Yves Le Traon. 2015. Trivial Compiler Equivalence: A Large Scale Empirical Study of a Simple, Fast and Effective Equivalent Mutant Detection Technique. In 37th *International Conference on Software Engineering (ICSE 2015)*. Florence, Italy, 936–946.

[69] Justyna Petke, Saemundur O. Haraldsson, Mark Harman, William B. Langdon, David R. White, and John R. Woodward. 2018. Genetic Improvement of Software: a Comprehensive Survey. *IEEE Transactions on Evolutionary Computation* 22, 3 (June 2018), 415–432.

[70] Juan Altmayer Pizzorno and Emery D Berger. 2024. Coverup: Coverage-guided LLM-based test generation. *arXiv preprint arXiv:2403.16218* (2024).

[71] Outi Räihä. 2010. A survey on Search–Based Software Design. *Computer Science Review* 4, 4 (2010), 203–249.

[72] Aurora Ramirez, Jose Raul Romero, and Christopher L Simons. 2018. A systematic review of interaction in search-based software engineering. *IEEE Transactions on Software Engineering* 45, 8 (2018), 760–781.

[73] Aurora Ramirez, José Raúl Romero, and Sebastian Ventura. 2019. A survey of many-objective optimisation in search-based software engineering. *Journal of Systems and Software* 149 (2019), 382–395.

[74] David Schuler and Andreas Zeller. 2009. Javalanche: efficient mutation testing for Java. In 7th *joint meeting of the European Software Engineering Conference and the ACM SIGSOFT International Symposium on Foundations of Software Engineering (ESEC/FSE 2009)*. 297–298.

[75] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: a concolic unit testing engine for C. In 10th *European Software Engineering Conference and 13th ACM International Symposium on Foundations of Software Engineering (ESEC/FSE '05)*, Michel Wermelinger and Harald Gall (Eds.). ACM, 263–272.

[76] Christopher L. Simons, Ian C. Parmee, and Rhys Gwynllyw. 2010. Interactive, Evolutionary Search in Upstream Object-Oriented Class Design. *IEEE Transactions on Software Engineering* 36, 6 (2010), 798–816.

[77] John Steven, Pravir Chandra, Bob Fleck, and Andy Podgurski. 2000. jRapture: A capture/replay tool for observation-based testing. In *Proceedings of the 2000 ACM SIGSOFT international symposium on Software Testing and Analysis (ISSTA 2000)*. 158–167.

[78] Deepika Tiwari, Martin Monperrus, and Benoit Baudry. 2023. Mimicking Production Behavior with Generated Mocks. arXiv:2208.01321 [cs.SE]

[79] Shreshth Tuli, Kinga Bojarczuk, Natalija Gucevska, Mark Harman, Xiao-Yu Wang, and Graham Wright. 2023. Simulation-Driven Automated End-to-End Test and Oracle Inference. In *45th IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, SEIP@ICSE 2023, Melbourne, Australia, May 14-20, 2023*. IEEE, 122–133.

[80] Lars van Hijfte and Ana Oprescu. 2021. Mutantbench: an equivalent mutant problem comparison framework. In *2021 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*. IEEE, 7–12.

[81] Margus Veanes, Colin Campbell, Wolfram Schulte, Pushmeet Kohli, N Tillmann, and W Grieskamp. 2005. On-the-fly testing of reactive systems. *Submitted for publication* (2005).

[82] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2023. Software testing with large language model: Survey, landscape, and vision. *arXiv preprint arXiv:2307.07221* (2023).

[83] Zejun Wang, Kaibo Liu, Ge Li, and Zhi Jin. 2024. HITS: High-coverage LLM-based Unit Test Generation via Method Slicing. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. 1258–1268.

[84] Michal Zalewski. Accessed March 27th 2025. American fuzzy lop. http://lcamtuf.coredump.cx/afl/

[85] Shuyin Zhao. 2023. GitHub Copilot now has a better AI model and new capabilities. https://github.blog/2023-02-14-github-copilot-now-has-a-better-ai-model-and-new-capabilities/

[86] Yuxiang Zhu and Minxue Pan. 2019. Automatic code summarization: A systematic literature review. *arXiv preprint arXiv:1909.04352* (2019).